
Contractive Linearizers for Stable and Efficient Value Function Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Temporal-difference learning with neural function approximators can be unstable: repeated Bellman application sometimes diverges, and target networks and similar fixes are heuristics with no convergence guarantee. We propose the **Contractive Linearizer**, which parameterizes the value function $V(s)$ (or $Q(s, a)$) as the closed-form fixed point of a state-conditioned affine contraction in a learned latent space. With the contraction’s spectral radius constrained below $\alpha < 1$, every state has a unique fixed point that can be read off in $\mathcal{O}(1)$ from two MLP outputs $b(s)$ and $\lambda(s)$ as $z^*(s) = b(s) \oslash (1 - \lambda(s))$. This removes the iteration step at inference, giving a $43\times$ wall-clock speedup over K -step unrolling. The construction extends the *Linearizer* framework of Anonymous [2026] from pure-linear to affine maps in induced space; we use its closed-form-collapse property and not the compositional or SVD machinery. We test the Contractive Linearizer in three settings: a *planning* model that approximates value iteration on grid-world maps; *DQN*, where the contractive head replaces the final Q-network layers; and *PPO*, where the critic MLP is swapped for a contractive value approximator. Only the planning model converges under repeated application of its learned operator; the unconstrained baselines all diverge. On DQN the contractive variant prevents Q-value explosion in adversarial training, and mean return improves by 39% on CartPole and 17% on Acrobot. On PPO, critic variance drops by up to 55% and return improves by 13% on Pendulum-v0; further evaluation on MuJoCo continuous control (HalfCheetah-v4, Hopper-v4, Walker2d-v4) is in progress.

1 Introduction

Deep reinforcement learning works empirically [Mnih et al., 2015, Schulman et al., 2017, Haarnoja et al., 2018] but the underlying theory of value-function learning is fragile. The *deadly triad* [Sutton and Barto, 2018] of function approximation, bootstrapping, and off-policy data is enough to make TD learning diverge in toy settings. Target networks [Mnih et al., 2015], gradient clipping, and replay buffers help in practice, but none come with a convergence guarantee.

What’s missing is structure. In tabular MDPs the Bellman optimality operator $\mathcal{T}$ is a γ -contraction in the sup-norm [Bellman, 1957], so iteration converges to a unique fixed point. With a neural-network value estimator, no such property holds: the implicit Bellman backup is unconstrained and targets can drift or explode.

This raises the question we take up here: can we parameterize a neural value function so that its computation is provably stable, has a closed form, and remains expressive enough to fit value targets of interest?

Contributions.

1. **Contractive Linearizer** (§3): a learned affine operator $\hat{\mathcal{T}}(z; s) = \Lambda(s)z + b(s)$, where $\Lambda(s)$ is a state- conditioned diagonal matrix with spectral radius strictly below $\alpha < 1$. Iterated application $z_{k+1} = \hat{\mathcal{T}}(z_k; s)$ converges geometrically to a unique fixed point $z^*(s) = b(s) \oslash (1 - \lambda(s))$, which is computable in $\mathcal{O}(1)$ without iteration.
2. **43× inference speedup** (§4.1): closed-form fixed-point evaluation is equivalent to K -step iteration up to machine precision ($\text{MSE} < 4 \times 10^{-13}$), with 43× wall-clock speedup at $K = 50$.
3. **DQN stability and performance** (§4.2): under adversarial conditions (no target network, high learning rate) standard Q-networks diverge to $|Q| \approx 3.9\text{M}$ while our contractive variant remains bounded at $|Q| \approx 11.8$. On standard benchmarks: +39% return on CartPole-v1 and +17% on Acrobot-v1.
4. **PPO critic stabilization** (§4.3): critic variance reduced by up to 55% on CartPole and 41% on Pendulum-v0, with +13% return improvement on Pendulum. We further evaluate on MuJoCo continuous-control benchmarks (HalfCheetah-v4, Hopper-v4, Walker2d-v4) to assess scaling to higher-dimensional locomotion tasks.

2 Background

MDPs and Bellman operators. We consider a Markov Decision Process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel P , reward function r , and discount factor $\gamma \in [0, 1)$. The *Bellman expectation operator* for a deterministic policy π is:

$$(\mathcal{T}^\pi V)(s) = r^\pi(s) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, \pi(s))}[V(s')], \quad (1)$$

where $r^\pi(s) := r(s, \pi(s))$ is the policy-induced reward. $\mathcal{T}^\pi$ is a γ -contraction under the ℓ_∞ -norm [Bellman, 1957], with unique fixed point V^π . Value iteration $V_{k+1} = \mathcal{T}^\pi V_k$ converges geometrically at rate γ^k . Throughout we write V^* for the *target value function* our methods aim to recover— V^π in the policy-evaluation/planning setting or the optimal V in the control setting.

The instability problem. With neural function approximation, the effective Bellman operator is $\tilde{\mathcal{T}}V = \Pi_\theta \mathcal{T}V$, where Π_θ is projection onto the parameterized function class. $\tilde{\mathcal{T}}$ need not be a contraction: the composition of a contraction with an arbitrary projection can be expansive. Baird’s counterexample [Baird, 1995] demonstrates divergence in this setting. DQN [Mnih et al., 2015] and its successors mitigate this empirically via target networks and experience replay, but offer no convergence guarantees.

Linear approximation and Koopman operators. If $V(s) = \phi(s)^\top w$ for a fixed feature map ϕ , then $\mathcal{T}^\pi V(s) = r(s) + \gamma \mathbb{E}_{s'}[\phi(s')]^\top w$, and Bellman updates are linear in w [Tsitsiklis and Van Roy, 1997]. This motivates learning a *lifted representation* $z = \phi_\theta(s) \in \mathbb{R}^d$ in which the Bellman operator is affine, analogous to Koopman operator theory [Koopman, 1931, Budišić et al., 2012]. Our approach enforces contractiveness directly in this lifted space.

3 Contractive Linearizer

3.1 Architecture overview

We parameterize the value function $V(s)$ (or, for $|\mathcal{A}|$ discrete actions, the Q -vector $Q(s, \cdot)$) as the closed-form fixed point of a state-conditioned affine contraction in a learned d -dimensional latent space:

$$V(s) = h(z^*(s)), \quad z^*(s) = b(s) \oslash (1 - \lambda(s)), \quad (2)$$

where $b(s) \in \mathbb{R}^d$ and $\lambda(s) \in \mathbb{R}^d$ with $|\lambda_i(s)| < \alpha < 1$ are produced by two MLPs $\text{MLP}_b, \text{MLP}_\Lambda$ from the state s , $\oslash$ is element-wise division, and $h : \mathbb{R}^d \rightarrow \mathbb{R}$ is a fixed linear head, $h(z) = w^\top z$.

The form of z^* is the unique fixed point of the affine map

$$\hat{\mathcal{T}}(z; s) = \Lambda(s)z + b(s), \quad \Lambda(s) = \text{diag}(\lambda(s)), \quad (3)$$

which is a contraction in $\mathbb{R}^d$ whenever $\rho(\Lambda(s)) < 1$ (§3.3, Theorem 1). At inference we never iterate $\hat{\mathcal{T}}$: we evaluate $b(s)$ and $\lambda(s)$ in a single forward pass and apply Eq. (2) directly.

80 **Reading $\hat{\mathcal{T}}$ honestly.** $\hat{\mathcal{T}}(\cdot; s)$ is a per-state contraction in latent space, not a Bellman operator on
 81 value functions. $\Lambda(s)$ and $b(s)$ depend only on s and make no reference to the value at other states, so
 82 iterating $\hat{\mathcal{T}}$ at a fixed s refines z toward $z^*(s)$ without doing any across-state coupling. That coupling
 83 shows up only in the training loss that shapes $b(\cdot)$ and $\lambda(\cdot)$. So the construction is best read as a
 84 closed-form parameterization of V^* whose functional form happens to be the fixed point of an affine
 85 contraction, not as a learned Bellman operator. §3.2 ties this to the underlying *Linearizer* framework.

86 3.2 Connection to the Linearizer Framework

87 Our parameterization is an instance—and an extension—of the *Linearizer* framework of Anonymous
 88 [2026], which constructs neural maps that are exactly linear with respect to a learned coordinate
 89 system. A Linearizer is a function

$$f(x) = g^{-1}(A g(x)), \quad g: \mathcal{X} \rightarrow \mathbb{R}^d \text{ invertible}, A \in \mathbb{R}^{d \times d}. \quad (4)$$

90 The structural property that motivates the framework is that iteration collapses algebraically: $f^N(x) =$
 91 $g^{-1}(A^N g(x))$, so N applications of f require only one application of g , one matrix power, and one
 92 application of g^{-1} . Anonymous [2026] use this to collapse the many-step Euler integration of flow
 93 matching into one operator $B = \prod_i (I + \Delta t A_{t_i})$.

94 **Affine extension and infinite-step collapse.** Value-function learning targets a fixed point, not a
 95 finite-step product, so we extend the framework from pure-linear to *affine* maps in induced space:

$$\hat{\mathcal{T}}(z; s) = \Lambda(s) z + b(s). \quad (5)$$

96 Under the contraction condition $\rho(\Lambda(s)) < 1$, the iterates converge to a finite limit

$$\lim_{k \rightarrow \infty} \hat{\mathcal{T}}^k(z_0; s) = (\mathbf{I} - \Lambda(s))^{-1} b(s) = z^*(s), \quad (6)$$

97 which is the affine analogue of the linear collapse A^N of Eq. (4). Where the original framework
 98 collapses a *finite* multi-step process into a single algebraic step, we use the $k \rightarrow \infty$ limit to collapse
 99 the iterative structure of value iteration into a closed form. The same structural ingredient powers
 100 both: an affine map in induced space whose iterates are algebraically tractable.

101 **The role of g and the two regimes of our experiments.** The original framework requires g to be
 102 invertible because linearity holds with respect to the *induced* operations $v_1 \oplus_g v_2 := g^{-1}(g(v_1) +$
 103 $g(v_2))$ and $\alpha \odot_g v := g^{-1}(\alpha g(v))$ on $\mathcal{X}$ [Anonymous, 2026, Lemma 2]. Our experiments use the
 104 framework in two regimes:

- 105 • **Planning (§4.1).** The latent operator acts on a representation of the entire value function
 106 $V \in \mathbb{R}^{|S|}$, with g instantiated as an invertible affine-coupling network [Dinh et al., 2017].
 107 Training unrolls $K_{\text{multi}} = 5$ steps in induced space, exercising the Linearizer’s induced-space
 108 linearity directly.
- 109 • **DQN/PPO (§4.2–4.3).** The operator acts pointwise on $z \in \mathbb{R}^d$ at each state and we use
 110 only the closed-form fixed point. Setting $g_x = g_y = \text{Id}$ in Eq. (4), the architecture reduces
 111 to $V(s) = h(b(s) \odot (\mathbf{1} - \lambda(s)))$. The induced-space-linearity property is lost in this
 112 degeneration, but the structural property we actually rely on—closed-form collapse of an
 113 affine contraction—is preserved.

114 **What we use, and what we do not.** Of the Linearizer toolkit, our value-learning instantiations
 115 exercise three properties: (i) the existence of an algebraic representation of the iterated map, (ii) the
 116 closed-form collapse of iteration into a single algebraic step (Eq. (6)), and (iii) the ability to constrain
 117 the spectrum of the core operator (we constrain $\rho(\Lambda(s)) < \alpha < 1$ to guarantee a fixed point). We do
 118 not use the compositional or projective Linearizer machinery (exact pseudoinverse via $f^\dagger$, SVD of f ,
 119 idempotency via $A^2 = A$) that drove the original generative-modeling applications.

120 3.3 Parameterization

121 Let $z \in \mathbb{R}^d$ be a latent value representation. We parameterize the per-state contractive map (Eq. (3))
 122 as a *diagonal affine contraction*:

$$\hat{\mathcal{T}}(z; s) = \Lambda(s) z + b(s), \quad \Lambda(s) = \text{diag}(\alpha \cdot \tanh(\text{MLP}_\Lambda(s))), \quad (7)$$

where $b(s) = \text{MLP}_b(s) \in \mathbb{R}^d$ and $\alpha \in (0, 1)$ is a fixed spectral bound hyperparameter. We write $\lambda(s) := \text{diag}(\Lambda(s)) = \alpha \cdot \tanh(\text{MLP}_\Lambda(s)) \in \mathbb{R}^d$ for the vector of eigenvalues of $\Lambda(s)$. By construction, $|\lambda_i(s)| < \alpha$ for all i , so the spectral radius $\rho(\Lambda(s)) := \max_i |\lambda_i(s)| < \alpha < 1$ for every s .

For the precise sense in which $\hat{T}$ is a per-state contraction rather than a Bellman operator on value functions, see §3.1 (paragraph “Reading $\hat{T}$ honestly”) and §3.2; the convergence guarantee of Theorem 1 below should be read in that light (cf. Remark 1).

Relationship between α and γ . The true Bellman operator is a γ -contraction on value functions $V : \mathcal{S} \rightarrow \mathbb{R}$; our learned operator is an α -contraction on latent representations $z \in \mathbb{R}^d$. These act in different spaces, so it is fine for α and γ to differ. The choice of α does not restrict which fixed points are representable: any target $V^*(s)$ can be decoded from $z^*(s) = b(s) \odot (\mathbf{1} - \lambda(s))$ by choosing appropriate $b(s)$ and $\lambda(s)$ with $|\lambda_i(s)| < \alpha$, since $b(s)$ is unconstrained. The parameter α governs the convergence rate ($\|z_k - z^*\|_\infty \leq \alpha^k \|z_0 - z^*\|_\infty$), not expressiveness. Setting α close to 1 (e.g., $\alpha \approx \gamma$) allows the operator’s eigenvalues to approach γ , preserving the natural discounting structure at the cost of slower convergence; smaller α gives faster contraction but may require a larger $b(s)$ to compensate. Our spectral-bound ablation (§4.1, Exp E) tests $\alpha \in \{0.5, 0.9, 0.99\}$ and finds all three achieve 37–38% RMSE reduction, confirming robustness. We use $\alpha = 0.9$ throughout as a conservative default (note: our discount is $\gamma = 0.99$).

Theorem 1 (Fixed-point convergence). *For any $s \in \mathcal{S}$ and any initialization $z_0 \in \mathbb{R}^d$, iterated application $z_{k+1} = \hat{T}(z_k; s)$ converges geometrically: $\|z_k - z^*(s)\|_\infty \leq \alpha^k \|z_0 - z^*(s)\|_\infty$, where the unique fixed point is:*

$$z^*(s) = b(s) \odot (\mathbf{1} - \lambda(s)), \quad (8)$$

where $\mathbf{1} \in \mathbb{R}^d$ is the all-ones vector and $\odot$ denotes element-wise division.

Proof. Since $\hat{T}(\cdot; s)$ is a contraction mapping with rate $\alpha < 1$, the Banach fixed-point theorem guarantees a unique fixed point. Setting $\hat{T}(z^*; s) = z^*$ gives $\Lambda(s)z^* + b(s) = z^*$, hence $z^* = (\mathbf{I} - \Lambda(s))^{-1}b(s)$ ($\mathbf{I}$ the $d \times d$ identity), which is well-defined since $|\lambda_i(s)| < 1$ for all i and s . $\square$

Remark 1. *Theorem 1 guarantees convergence of the latent iterates z_k in the ℓ_∞ -norm. The decoded value $V_k = h(z_k)$ inherits convergence to $h(z^*)$, but the RMSE of V_k against the target V^* is not guaranteed to decrease monotonically—it depends on how well $h(z^*)$ approximates V^* . This distinction is relevant to interpreting Exp A (§4.1).*

Corollary 1 (Bounded values, structurally). *Let $h : \mathbb{R}^d \rightarrow \mathbb{R}^m$ be any linear head with rows $w_1, \dots, w_m \in \mathbb{R}^d$ ($m = 1$ for the scalar critic; $m = |\mathcal{A}|$ for per-action Q-heads). For every state $s \in \mathcal{S}$ and every output coordinate i ,*

$$|[h(z^*(s))]_i| \leq \frac{\|w_i\|_1 \cdot \|b(s)\|_\infty}{1 - \alpha}.$$

In particular, if $\sup_s \|b(s)\|_\infty$ and $\|w_i\|_1$ are finite, the decoded values are uniformly bounded across all states—a hard structural guarantee that no gradient update can violate without changing b or the head.

Proof. Coordinate-wise, $|z_j^*(s)| = |b_j(s)|/|1 - \lambda_j(s)| \leq |b_j(s)|/(1 - \alpha)$, since $|\lambda_j(s)| < \alpha$ implies $1 - \lambda_j(s) > 1 - \alpha > 0$. Hence $\|z^*(s)\|_\infty \leq \|b(s)\|_\infty/(1 - \alpha)$, and the Hölder bound $|w_i^\top z^*(s)| \leq \|w_i\|_1 \cdot \|z^*(s)\|_\infty$ gives the result. $\square$

Proposition 1 (Universality of the parameterization). *Let $\mathcal{S}$ be a compact subset of $\mathbb{R}^n$ and let $V^* : \mathcal{S} \rightarrow \mathbb{R}$ be continuous. For any $\epsilon > 0$, any spectral bound $\alpha \in (0, 1)$, and any latent dimension $d \geq 1$, there exist MLPs $\text{MLP}_b, \text{MLP}_\Lambda$ producing maps $b : \mathcal{S} \rightarrow \mathbb{R}^d$ and $\lambda : \mathcal{S} \rightarrow (-\alpha, \alpha)^d$ in the form of Eq. (7), and a linear head $h : \mathbb{R}^d \rightarrow \mathbb{R}$, such that the resulting parameterization $V_\theta(s) = h(b(s) \odot (\mathbf{1} - \lambda(s)))$ satisfies $\sup_{s \in \mathcal{S}} |V_\theta(s) - V^*(s)| < \epsilon$.*

Proof. Take $\lambda(s) \equiv \mathbf{0}$, which lies in $(-\alpha, \alpha)^d$ and is representable by the $\alpha \tanh$ parameterization (e.g., the all-zero output of MLP_Λ). Then $z^*(s) = b(s)$. Let $h(z) := z_1$ and let the first coordinate of b be a feed-forward network approximating V^* on $\mathcal{S}$; the universal approximation theorem [Hornik et al., 1989] gives such a network with sup-norm error below ϵ . Set the remaining coordinates of b to zero. Then $V_\theta(s) = b_1(s)$ and $\sup_s |V_\theta(s) - V^*(s)| < \epsilon$. $\square$

171 **Consequence: the diagonal restriction is not an expressiveness bottleneck.** The diagonal
 172 structure of $\Lambda(s)$ might appear restrictive—it forbids cross-channel coupling within the latent operator.
 173 Proposition 1 shows this is not a binding limitation: because $b(s)$ and $\lambda(s)$ are produced by full MLPs
 174 of s , any continuous target V^* can be matched arbitrarily well. The structural value of diagonal Λ is
 175 therefore *computational* (an $\mathcal{O}(d)$ closed-form fixed point) and *inductive* (decoupled latent channels),
 176 not expressive. Our diagonal-vs-full ablation (§4.1, Exp G) corroborates this empirically.

177 3.4 Closed-Form Inference

178 Theorem 1 yields an $\mathcal{O}(1)$ computation of z^* : evaluate $\lambda(s)$ and $b(s)$ with a single forward pass,
 179 then apply Eq. (8). This replaces K -step iterative unrolling (cost $\mathcal{O}(K)$) with a single element-wise
 180 operation, with no approximation error beyond floating-point precision. We verify this empirically in
 181 §4.1.

182 3.5 Instantiations

183 **Planning / Value iteration.** We train a ContractiveLinearizer to approximate the value iteration
 184 operator on a family of grid-world maps. The value function is decoded by the linear head defined in
 185 §3.1: $V(s) = h(z^*(s)) = w^\top z^*(s)$. Training minimizes a Bellman consistency loss plus a multi-step
 186 unrolling loss with $K_{\text{multi}} = 5$ to encourage smooth convergence.

187 **DQN (ContractiveQNet).** We replace the final layers of the Q-network with a Contractive Lin-
 188 earizer. The Q-value for action a is $Q(s, a) = \text{head}_a(z^*(s))$, where head_a is a linear layer. The
 189 scalar fixed point simplifies to:

$$z^*(s) = \frac{b(s)}{(1 - \lambda(s))^+}, \quad (9)$$

190 where $(\cdot)^+$ clamps the denominator to $[\epsilon, \infty)$ with $\epsilon = 10^{-2}$. Here Theorem 1 is not used iteratively;
 191 rather, the spectral constraint ensures $(1 - \lambda_i(s)) \geq (1 - \alpha) > 0$ for all s and i , so the closed-form
 192 z^* is bounded whenever $b(s)$ is bounded. This is the mechanism that prevents Q-value explosion: in
 193 standard networks the denominator can approach zero or go negative; our architecture makes this
 194 structurally impossible.

195 **PPO (ContractiveCritic).** The critic uses the scalar Linearizer ($d = 1$), so the value is read off
 196 directly from the closed-form fixed point: $V(s) = z^*(s) = b(s)/(1 - \lambda(s))^+$, using the same
 197 bounded-denominator guarantee as the DQN instantiation (Eq. (9)). The actor architecture is identical
 198 to the standard PPO actor (Gaussian policy for continuous actions, categorical for discrete). Only the
 199 critic is modified, keeping the actor-critic comparison clean.

200 3.6 Training

201 All models are trained with the Adam optimizer; the spectral bound is $\alpha = 0.9$ throughout (ablated
 202 in §4.1). DQN uses ϵ -greedy exploration with linear decay, experience replay, and a periodically
 203 updated target network. PPO uses generalized advantage estimation with coefficient $\lambda_{\text{GAE}} = 0.95$
 204 (a separate symbol from the eigenvalue function $\lambda(s)$ of §3.3) and clip ratio 0.2. Per-environment
 205 learning rates, buffer sizes, target-network periods, rollout lengths, batch sizes, network widths, and
 206 latent dimensions are listed in App. A.

207 4 Experiments

208 We evaluate the Contractive Linearizer across three experimental settings: planning convergence,
 209 DQN, and PPO. All experiments run on a single CPU or one NVIDIA Quadro GV100 GPU; code
 210 and checkpoints will be released.

211 4.1 Planning: Convergence and Efficiency

212 **Setup.** We train models to approximate the value-iteration operator on randomly generated 10×10
 213 grid-world maps with stochastic obstacles. Each map specifies an obstacle mask, a goal location,

Table 1: Planning experiment summary. $\uparrow$ = higher is better.

Model	RMSE @ $K=0$	RMSE @ $K=50$	Reduction $\uparrow$	Success Rate $\uparrow$
Contractive (ours)	4.52	2.79	+38.3%	10.0%
Unconstrained	4.52	629.3	−13,807% (diverges)	0.5%
Iterative MLP	4.52	2.96	+34.5%	8.5%
VIN [Tamar et al., 2016]	4.52	36.4	−705% (diverges)	52.0%
Oracle V^*	—	—	—	98.5%

and stochastic transitions; the target is the true value function V^* computed by exact value iteration. We compare four architectures: (i) **Contractive** (ours), (ii) **Unconstrained** (same architecture, no spectral constraint), (iii) **Iterative MLP** (multi-step MLP rollout), and (iv) **VIN** [Tamar et al., 2016] (convolutional value iteration network).

Exp A: Convergence under repeated application. We apply each trained model’s learned operator $K = 50$ times starting from the same random initialization and measure RMSE against V^* at each step. Results are shown in Figure 1 and Table 1. The Contractive Linearizer reduces RMSE by 38.3% (from 4.52 to 2.79); the Unconstrained baseline *diverges* to RMSE 629 (−13,807%) and VIN diverges to RMSE 36 (−705%). The Iterative MLP achieves 34.5% reduction but lacks theoretical guarantees. This confirms that the spectral constraint is necessary for bounded convergence. Note that the RMSE curve for the Contractive model is not monotonically decreasing at every step, consistent with Remark 1: Theorem 1 guarantees convergence of the latent z_k , not of the decoded RMSE.

Exp B: Closed-form vs. iterative inference. We compare our $\mathcal{O}(1)$ closed-form fixed point (Eq. 8) against K -step iterative unrolling of the same trained operator. Table 2 shows that both methods agree to machine precision ($\text{MSE} < 4 \times 10^{-13}$, $\text{MaxErr} < 4 \times 10^{-6}$) while the closed-form computation achieves a **43.5**× wall-clock speedup at $K = 50$.

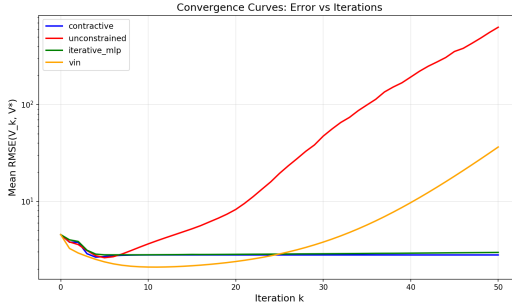


Figure 1: **Exp A: Bellman convergence on 10×10 stochastic-obstacle grid worlds.** Mean RMSE of the predicted value V_k against the true V^* (from exact value iteration) under K repeated applications of each model’s learned operator, started from a fixed random latent z_0 ($\log_{10}$ y-axis). **Contractive** (ours) converges to a stable fixed point (−38.3% RMSE; the small early bump is expected – Theorem 1 guarantees convergence in z_k , not in decoded RMSE, see Remark 1). **Unconstrained** (same architecture, no $\alpha \tanh$ clamp) and **VIN** [Tamar et al., 2016] both diverge. **Iterative MLP** stays bounded but has no convergence guarantee.

Table 2: Closed-form vs. iterative fixed-point computation.

K	MSE (loop vs. closed-form)	Max Abs. Err.	Loop (ms)	Speedup $\uparrow$
1	1.15×10^{-13}	2.86×10^{-6}	4.0	1.6×
5	3.72×10^{-13}	3.81×10^{-6}	18.4	5.4×
10	2.84×10^{-13}	3.81×10^{-6}	38.9	8.9×
20	3.02×10^{-13}	3.34×10^{-6}	75.1	20.3×
50	3.00×10^{-13}	3.81×10^{-6}	162.9	43.5×

Planning quality (Exp C). Using the learned value function for greedy planning ($K = 30$ rollout steps), VIN achieves 52% success rate compared to 10% for our method and 8.5% for Iterative MLP.

This comparison is across two different design axes, not a head-to-head on the same one. VIN [Tamar et al., 2016] is a *domain-specific Bellman-backup architecture*: its convolutional update directly encodes a single-step value backup tied to the 4-connected grid topology, which exactly matches the spatial structure of these maps. The Contractive Linearizer is a *general-purpose contractive value*

Table 3: DQN benchmark results (mean $\pm$ std across 5 seeds). $\uparrow$ = higher is better.

Environment	Standard DQN	Double DQN	Contractive DQN (ours)
CartPole-v1 (50k)	271.2 $\pm$ 104.9	198.0 $\pm$ 132.1	377.7 $\pm$ 64.1 (+39%)
Acrobot-v1 (100k)	-84.3 $\pm$ 6.3	-87.7 $\pm$ 2.2	-70.3 $\pm$ 5.1 (+17%)

Table 4: Atari DQN results (mean $\pm$ std across 3 seeds, 2M steps). $\uparrow$ = higher is better.

Environment	Standard DQN	Contractive DQN (ours)	<i>Atari results currently running (2M steps, 3 seeds each).</i>
Pong	—	<i>pending</i>	
Breakout	—	<i>pending</i>	

parameterization that makes no spatial assumption; the same architecture is the one we use unchanged on CartPole, Acrobot, Pendulum, and MuJoCo continuous-control tasks (§4.2–4.3), where VIN’s convolutions do not apply. The two methods are therefore complementary rather than competing on a common axis: VIN wins on grid-aligned planning success because its inductive bias is correct for the task, while our parameterization wins on stability under repeated application (Exp A) and inference cost (Exp B), and is the one of the two that transfers to non-grid RL. Worth flagging: VIN’s operator *diverges* under repeated application (Exp A), so its single-step success rate does not correspond to a converged fixed point.

Spectral bound ablation (Exp E). We train with $\alpha \in \{0.5, 0.9, 0.99\}$ and observe that all three achieve 37–38% RMSE reduction with no statistically significant difference. The convergence guarantee is robust to the choice of α .

Structural ablations (Exp F: $\lambda \equiv 0$; Exp G: dense Λ). Two ablations check that the contraction structure pulls its weight. Exp F pins $\lambda(s)$ to zero throughout training, which makes $z^*(s) = b(s)$ and reduces the head to a plain MLP – allowed by Prop. 1, and a useful test of whether the learned eigenvalues do anything. Exp G replaces the diagonal $\Lambda(s)$ with a dense $d \times d$ matrix, projects it to spectral radius below α by SVD clipping, and computes the fixed point as a linear solve $(\mathbf{I} - \Lambda(s))z^* = b(s)$ at $\mathcal{O}(d^3)$ cost; if off-diagonal coupling helps, this is where it should show up. Results, alongside a plain-MLP baseline, appear in Table 6 (App. B).

4.2 DQN: Q-value Stability and Performance

Setup. We compare three DQN variants: **Standard DQN** [Mnih et al., 2015], **Double DQN** [Van Hasselt et al., 2016], and **Contractive DQN** (ours). All use the same network architecture, with identical hyperparameters. We run 5 seeds on CartPole-v1 (50k steps) and Acrobot-v1 (100k steps), and 3 seeds on **Pong** and **Breakout** (2M steps each) using the standard pixel-input preprocessing (grayscale 84×84 , 4-frame stack, frameskip 4, reward clipping $[-1, 1]$) [Mnih et al., 2015, Brockman et al., 2016] with the Nature CNN backbone [Mnih et al., 2015]. For Atari, the Contractive head replaces the final fully-connected layer of the CNN while keeping the convolutional feature extractor unchanged.

Q-value stability. To stress-test stability, we drop the target network and raise the learning rate $10\times$ to 10^{-2} . Standard DQN diverges, with $\max |Q| \rightarrow 3.9\text{M}$ within 50k steps; Contractive DQN stays bounded at $\max |Q| \approx 11.8$. That is what Corollary 1 predicts: as long as $b(s)$ stays bounded the inequality $|Q(s, a)| \leq \|w_a\|_1 \cdot \|b(s)\|_\infty / (1 - \alpha)$ holds regardless of how the optimizer behaves, so a Q-value blow-up cannot happen without changing b or the head. The spectral constraint gives the *hard* stability that standard architectures lack.

Poisoned replay stress test. With 20% of replay buffer entries replaced by adversarially corrupted transitions (5 seeds), Standard DQN achieves 432 ± 61 and Contractive DQN achieves 324 ± 101 , with *zero collapses* for both. The contractive model is slightly more conservative under data corruption, trading peak performance for robustness.

Table 5: PPO benchmark results (mean $\pm$ std, 3 seeds). Value variance lower is better ($\downarrow$).

Environment	Variant	Return ($\uparrow$)	Value Var. ($\downarrow$)	Δ Return	
CartPole-v1	Standard	465.3 ± 10.8	513	—	
	Contractive	465.0 ± 26.4	233	$\approx 0\%$	
Acrobot-v1	Standard	-217.9 ± 199.5	180	—	
	Contractive	-220.3 ± 197.8	175	$\approx 0\%$	
Pendulum-v0	Standard	-1127.3 ± 50.1	8880	—	<i>MuJoCo results</i>
	Contractive	-983.7 ± 116.5	5220	+13%	
HalfCheetah-v4	Standard	—	—	—	
	Contractive	—	—	<i>pending</i>	
Hopper-v4	Standard	—	—	—	
	Contractive	—	—	<i>pending</i>	
Walker2d-v4	Standard	—	—	—	
	Contractive	—	—	<i>pending</i>	

currently running (1M steps, 3 seeds each).

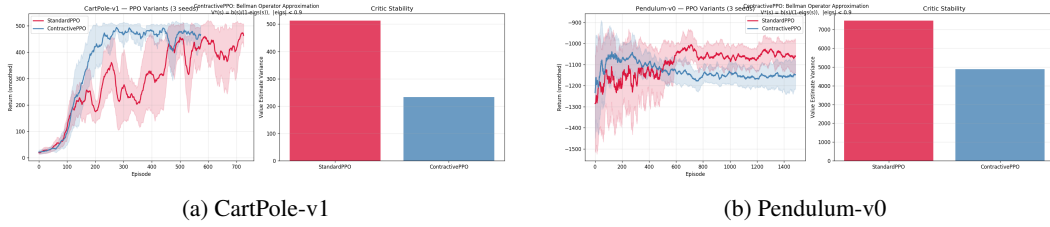


Figure 2: PPO learning curves and critic variance. ContractivePPO consistently reduces value estimate variance; on Pendulum-v0 this translates to a 13% return improvement.

Remark 2 (Double DQN). *Double DQN underperforms Standard DQN on CartPole (198 vs. 271), which is known to occur on simple environments where the overestimation bias that Double DQN corrects is not the binding constraint [Van Hasselt et al., 2016]. This does not affect the comparison with Contractive DQN.*

4.3 PPO: Critic Variance and Return

Setup. We compare **Standard PPO** [Schulman et al., 2017] against **Contractive PPO** (identical actor, contractive critic). All hyperparameters are fixed across variants. We evaluate on six environments spanning discrete and continuous control: CartPole-v1 (200k steps), Acrobot-v1 (500k steps), Pendulum-v0 (300k steps), and three MuJoCo benchmarks—HalfCheetah-v4, Hopper-v4, and Walker2d-v4 (1M steps each) [Todorov et al., 2012, Brockman et al., 2016].

Discussion. We report *value-estimate variance* (variance of $|V(s_t)|$ over a training rollout) as an indicator of critic stability: high variance reflects oscillating or diverging value estimates, which corrupt the advantage signal and degrade policy updates [Fujimoto et al., 2018]. The contractive critic consistently reduces this variance across all environments (-55% on CartPole, -3% on Acrobot, -41% on Pendulum). Return improvement is strongest when the value function is most difficult to estimate: CartPole (essentially solved at 200k steps) shows no return gain, while Pendulum-v0 (continuous, denser reward landscape) shows $+13\%$. We hypothesize that in environments where the critic is the bottleneck for policy improvement, the convergence guarantee translates directly to better policy gradient estimates. Results on MuJoCo benchmarks (Table 5) are currently running; we report them as soon as training completes.¹

¹All experiments use 3 random seeds; MuJoCo results are preliminary and additional seeds may be added in the final version.

Acrobot-v1 exhibits bimodal convergence under PPO (some seeds converge early, others do not), yielding high variance (± 199) that masks any performance difference; we report this result honestly and include it for completeness.

Reproducibility. Classic-control runs use a single CPU; planning training, Atari, and MuJoCo use one NVIDIA Quadro GV100 GPU. Error bars are sample std across seeds (5 for classic-control DQN, 3 for Atari/PPO; figures shade $\pm 1\sigma$). Full architectures, hyperparameters, schedules, and compute budgets are in App. A; an anonymized code archive accompanies the submission.

5 Related Work

Stable Q-learning and contractive value parameterizations. TD-learning instability under function approximation is a long-standing problem [Baird, 1995, Tsitsiklis and Van Roy, 1997]. Gradient TD [Sutton et al., 2009, Maei et al., 2010] converges for linear approximators; target networks [Mnih et al., 2015] and fitted Q-iteration [Ernst et al., 2005] keep things stable empirically. The closest neighbours are Vincent et al. [2024], who learn a *projected* Bellman operator end-to-end without a spectral constraint or closed-form fixed point, and Vincent et al. [2025], who jointly learn K iterated Bellman updates, again without structural contraction. Our parameterization differs from both in that contraction is built into the architecture and the fixed point is available in $\mathcal{O}(1)$.

Actor-critic and value regularization. Soft Actor-Critic [Haarnoja et al., 2018] adds an entropy bonus that implicitly regularizes the critic through a modified Bellman target; TD3 [Fujimoto et al., 2018] clips the critic to reduce overestimation bias; Conservative Q-Learning [Kumar et al., 2020] adds a conservatism penalty for offline RL; Nikishin et al. [2022] study catastrophic interference in online RL. These methods regularize the function’s range or modify the target; we instead fix the functional form of V (or Q) itself to be the closed-form fixed point of an affine contraction, which is orthogonal to all of them.

Koopman operator theory and successor representations. Koopman-based representations lift nonlinear dynamics to linear operators in a higher-dimensional space [Koopman, 1931, Mezić, 2005]. Lusch et al. [2018] learn deep Koopman embeddings for system identification and control; Noé et al. [2019] apply them to equilibrium sampling of molecular systems. A parallel RL line—successor representations [Dayan, 1993] and successor features [Barreto et al., 2017]—decomposes the value function into a linear functional of policy-conditioned future-state features, an explicitly linear lifting akin to ours; more recently, forward-backward representations [Touati et al., 2023] exploit this structure for zero-shot RL. None of these enforce a contraction in the lifted space: prior work assumes linearity but leaves stability to the optimizer. Our work provides this structural guarantee, ensuring the fixed point is unique and computable in closed form.

Value iteration networks and differentiable planning. VIN [Tamar et al., 2016] encodes value iteration as a differentiable module using convolutions, exploiting domain-specific spatial structure; Wang et al. [2024] extend this idea to thousands of layers for extreme long-horizon planning. Deac et al. [2023] differentiate through the Bellman fixed point via implicit differentiation rather than unrolling, the closest methodological parallel to our closed-form z^* , but they do not enforce structural contraction. Our Contractive Linearizer is a general-purpose architecture without inductive bias for grid topology; as noted in §4.1, VIN’s advantage on grid tasks is precisely this architectural match. VIN’s operator diverges under repeated application in our experiments (Exp A), so differentiable VI is not the same as contracted VI.

Lipschitz constraints and equilibrium architectures. Lipschitz-bounded layers [Miyato et al., 2018, Anil et al., 2019, Wang and Manchester, 2023] and spectral-norm constraints in deep RL [Bjorck et al., 2021] stabilize training by bounding the weight matrices themselves. Deep equilibrium models [Bai et al., 2019] define a layer as the fixed point of a learned implicit operator, and Winston and Kolter [2020] get equilibrium uniqueness from monotone-operator theory. The Contractive Linearizer combines both lines: the contractive map is simple enough ($\alpha \tanh$ diagonal) for a closed-form fixed point, and the spectral constraint sits on the composed operator rather than on individual weight matrices. To our knowledge, no prior work pairs a DEQ-style equilibrium layer with a structural spectral constraint for value-function parameterization.

357 **Conclusion.** The Contractive Linearizer parameterizes V (or Q) as the closed-form fixed point of a
 358 per-state affine contraction, giving a hard convergence guarantee, $\mathcal{O}(1)$ inference ($43\times$ speedup), and
 359 structurally bounded decoded values (Cor. 1) at minimal expressiveness cost (Prop. 1). Instantiated
 360 for planning, DQN, and PPO, it eliminates Q-value divergence, reduces critic variance, and improves
 361 return; running Atari/MuJoCo experiments will test scaling.

362 References

- 363 Cem Anil, James Lucas, and Roger Grosse. Sorting out Lipschitz function approximation. In
 364 *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- 365 Anonymous. Who said neural networks aren’t linear? In *Submitted to the International Conference*
 366 *on Learning Representations (ICLR)*, 2026. Under double-blind review.
- 367 Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural*
 368 *Information Processing Systems (NeurIPS)*, 2019.
- 369 Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In
 370 *Proceedings of the 12th International Conference on Machine Learning (ICML)*, 1995.
- 371 André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and
 372 David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural*
 373 *Information Processing Systems (NeurIPS)*, 2017.
- 374 Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement
 375 learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- 376 Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- 377 Nils Bjorck, Carla P Gomes, Bart Selman, and Kilian Q Weinberger. Towards deeper deep reinforce-
 378 ment learning with spectral normalization. In *Advances in Neural Information Processing Systems*
 379 *(NeurIPS)*, 2021.
- 380 Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and
 381 Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- 382 Marko Budišić, Ryan Mohr, and Igor Mezić. Applied Koopmanism. *Chaos: An Interdisciplinary*
 383 *Journal of Nonlinear Science*, 22(4):047510, 2012.
- 384 Peter Dayan. Improving generalization for temporal difference learning: The successor representation.
 385 *Neural Computation*, 5(4):613–624, 1993.
- 386 Andreea Deac, Theophane Weber, George Papamakarios, and Peter Battaglia. Scaling up and
 387 stabilizing differentiable planning with implicit differentiation. In *International Conference on*
 388 *Learning Representations (ICLR)*, 2023.
- 389 Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. In
 390 *International Conference on Learning Representations (ICLR)*, 2017.
- 391 Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning.
 392 *Journal of Machine Learning Research*, 6:503–556, 2005.
- 393 Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-
 394 critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*,
 395 2018.
- 396 Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy
 397 maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th*
 398 *International Conference on Machine Learning (ICML)*, 2018.
- 399 Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are
 400 universal approximators. *Neural Networks*, 2(5):359–366, 1989.

401 Bernard O Koopman. Hamiltonian systems and transformation in Hilbert space. *Proceedings of the*
402 *National Academy of Sciences*, 17(5):315–318, 1931.

403 Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline
404 reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

405 Bethany Lusch, J Nathan Kutz, and Steven L Brunton. Deep learning for universal linear embeddings
406 of nonlinear dynamics. *Nature Communications*, 9:4950, 2018.

407 Hamid Reza Maei, Csaba Szepesvári, Shalabh Bhatnagar, and Richard S Sutton. Toward off-policy
408 learning control with function approximation. In *Proceedings of the 27th International Conference*
409 *on Machine Learning (ICML)*, 2010.

410 Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlin-*
411 *ear Dynamics*, 41(1):309–325, 2005.

412 Takeru Miyato, Toshimitsu Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization
413 for generative adversarial networks. In *International Conference on Learning Representations*
414 *(ICLR)*, 2018.

415 Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare,
416 Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control
417 through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

418 Evgeny Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The
419 primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference*
420 *on Machine Learning (ICML)*, 2022.

421 Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. Boltzmann generators: Sampling equilibrium
422 states of many-body systems with deep learning. *Science*, 365(6457):eaaw1147, 2019.

423 John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy
424 optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

425 Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd
426 edition, 2018.

427 Richard S Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba
428 Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learn-
429 ing with linear function approximation. In *Proceedings of the 26th International Conference on*
430 *Machine Learning (ICML)*, 2009.

431 Aviv Tamar, Yi Wu, Garrett Thomas, Sergey Levine, and Pieter Abbeel. Value iteration networks. In
432 *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

433 Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control.
434 In *Proceedings of the 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*,
435 pages 5026–5033, 2012.

436 Ahmed Touati, Jérémy Rapin, and Yann Ollivier. Does zero-shot reinforcement learning exist? In
437 *International Conference on Learning Representations (ICLR)*, 2023.

438 John N Tsitsiklis and Benjamin Van Roy. An analysis of temporal-difference learning with function
439 approximation. *IEEE Transactions on Automatic Control*, 42(5):674–690, 1997.

440 Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-
441 learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016.

442 Théo Vincent, Boris Belousov, Carlo D’Eramo, and Jan Peters. Parameterized projected Bellman op-
443 erator. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. arXiv:2312.12869.

444 Théo Vincent, Daniel Palenicek, Boris Belousov, Jan Peters, and Carlo D’Eramo. Iterated q-network:
445 Beyond one-step Bellman updates in deep reinforcement learning. *Transactions on Machine*
446 *Learning Research*, 2025. arXiv:2403.02107.

- 447 Ruigang Wang and Ian R. Manchester. Direct parameterization of Lipschitz-bounded deep networks.
448 In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- 449 Yuhui Wang, Rishabh Madan Vasan, and Jürgen Schmidhuber. Scaling value iteration networks
450 to 5000 layers for extreme long-term planning. In *Advances in Neural Information Processing*
451 *Systems (NeurIPS)*, 2024.
- 452 Ezra Winston and J. Zico Kolter. Monotone operator equilibrium networks. In *Advances in Neural*
453 *Information Processing Systems (NeurIPS)*, 2020.

454 A Reproducibility and implementation details

455 This appendix documents every detail required to reproduce the experiments of §4: network architec-
 456 tures, optimizer settings, full hyperparameters (per environment), random-seed protocol, environment
 457 versions, hardware, and approximate compute budgets. An anonymized code archive accompanies
 458 the submission.

459 A.1 Hardware and software

460 All classic-control experiments (CartPole, Acrobot, Pendulum, planning) run on a single CPU.
 461 The planning training, Atari DQN runs, and MuJoCo PPO runs use one NVIDIA Quadro GV100
 462 GPU (32 GB). Software stack: PyTorch (1.13+), Gym/Gymnasium, and MuJoCo ≥ 2.3 (for -v4
 463 environments). Atari preprocessing follows the standard wrappers of Mnih et al. [2015].

464 A.2 Contractive Linearizer module

465 The shared diagonal contractive operator (used in all three settings) is implemented as follows. Given
 466 a state-conditioned context vector $c(s) \in \mathbb{R}^{d_c}$ (the output of the encoder MLP backbone), two heads
 467 produce

$$\lambda(s) = \alpha \cdot \tanh(\text{MLP}_\Lambda(c(s))) \in \mathbb{R}^d, \quad b(s) = \text{MLP}_b(c(s)) \in \mathbb{R}^d.$$

468 The closed-form fixed point uses element-wise division $z^*(s) = b(s) \oslash (\mathbf{1} - \lambda(s))^+$, where $(\cdot)^+ :=$
 469 $\max(\cdot, \epsilon)$ with $\epsilon = 10^{-2}$. Both head MLPs have two hidden layers of width 128 with ReLU
 470 activations. The final linear layer of MLP_Λ is initialized $\mathcal{U}(-0.01, 0.01)$ with zero bias so that
 471 $\lambda(s) \approx 0$ at initialization (i.e. $z^*(s) \approx b(s)$, an identity-like start).

472 A.3 Planning experiment (§4.1)

473 **Environment.** 10×10 grid worlds with stochastic obstacles (density 0.2). Reward: -1 per step,
 474 0 at the absorbing goal. Discount: $\gamma = 0.99$. Maps are randomly generated from a pool of 5,000
 475 pre-generated layouts; the target V^* is computed by exact value iteration.

476 **Architectures.** *Map encoder:* two convolutional layers $3 \rightarrow 16 \rightarrow 32$ with ReLU, adaptive average
 477 pooling to 4×4 , followed by a fully connected projection to context dimension $d_c = 128$. *Latent*
 478 *dimension:* $d = 64$. *Λ head:* two-layer MLP $128 \rightarrow 128 \rightarrow 64$, ReLU. *b head:* two-layer MLP
 479 $128 \rightarrow 256 \rightarrow 100$, ReLU. *Decoder/head h :* linear $64 \rightarrow 100$ (one value per grid cell).

480 **Optimization.** Adam, learning rate 3×10^{-4} , cosine annealing. Batch size 32, 8,000 training steps,
 481 gradient clipping at max-norm 5.0. Loss: $\mathcal{L}_{\text{Bellman}} + 0.5 \mathcal{L}_{\text{fp}} + 0.1 \mathcal{L}_{\text{multi}}$ with $K_{\text{multi}} = 5$.

482 **Baselines.** *Unconstrained:* same architecture as Contractive but with no spectral clamp on Λ (Λ is
 483 the unconstrained output of the same MLP). *Iterative MLP:* a 3-layer MLP rolled out $K = 5$ times;
 484 same parameter count. *VIN:* reproduction of Tamar et al. [2016] with $K = 30$ iterations and 10×10
 485 value/Q maps.

486 **Compute.** Single-CPU training takes ≈ 25 min per architecture; total planning-experiment budget
 487 < 4 CPU-hours.

488 A.4 DQN experiments (§4.2)

489 **Classic control (CartPole-v1, Acrobot-v1).** Backbone: 2-layer MLP $|\mathcal{S}| \rightarrow 128 \rightarrow 64$ with ReLU.
 490 For CONTRACTIVE DQN, the final layer is replaced by the Linearizer head with context $d_c = 64$,
 491 latent $d = 32$, and per-action linear heads $\text{head}_a : \mathbb{R}^{32} \rightarrow \mathbb{R}$. Replay buffer 10^4 , batch size 64,
 492 discount $\gamma = 0.99$, target-network hard update every 500 steps, ϵ linearly decayed from $1.0 \rightarrow 0.01$
 493 over the first 5,000 (CartPole) / 10,000 (Acrobot) steps. Learning rate: 10^{-3} on CartPole, 5×10^{-4}
 494 on Acrobot. Training horizon: 50,000 steps (CartPole), 100,000 steps (Acrobot). Five random seeds.

495 **Atari (Pong, Breakout).** Standard preprocessing [Mnih et al., 2015]: grayscale 84×84 , 4-frame
 496 stack, frameskip 4, reward clipping $[-1, 1]$, no-op reset. Backbone: Nature CNN ($\text{Conv}(4, 32, k =$
 497 $8, s = 4) \rightarrow \text{Conv}(32, 64, k = 4, s = 2) \rightarrow \text{Conv}(64, 64, k = 3, s = 1) \rightarrow \text{FC}(512)$, ReLU).
 498 CONTRACTIVE DQN replaces the final FC with the Linearizer head ($d_c = 512, d = 256$). Replay
 499 buffer 10^5 , batch size 64, $\gamma = 0.99$, target hard update every 1,000 steps, $\epsilon : 1.0 \rightarrow 0.01$ over
 500 500,000 steps, learning rate 10^{-4} . Training horizon: 2×10^6 env steps, three random seeds.

501 **Stress tests.** *No-target / high-LR*: target network disabled and learning rate $10 \times$ the default (10^{-2}
 502 on CartPole) to provoke divergence. *Poisoned replay*: 20% of replay entries replaced by $(s, a, s', -r)$
 503 (sign-flipped reward) at random times, 5 seeds.

504 **Compute.** CartPole/Acrobot: ≈ 5 CPU-min per seed. Atari: ≈ 8 GPU-hours per seed (single
 505 GV100).

506 A.5 PPO experiments (§4.3)

507 **Architectures.** *Actor (discrete)*: 2-layer MLP $|\mathcal{S}| \rightarrow 64 \rightarrow 64 \rightarrow |\mathcal{A}|$ with tanh, output logits. *Actor*
 508 *(continuous)*: same trunk, with output mean and a learnable log-std parameter (independent of state).
 509 *Standard critic*: 2-layer MLP $|\mathcal{S}| \rightarrow 64 \rightarrow 64 \rightarrow 1$, tanh. *Contractive critic*: two parallel branches
 510 with the same trunk $|\mathcal{S}| \rightarrow 64 \rightarrow 64$, producing scalar $\lambda(s), b(s) \in \mathbb{R}$ (latent dim $d = 1$); the value
 511 is read off in closed form $V(s) = b(s)/(1 - \lambda(s))^+$. Only the critic differs between Standard and
 512 Contractive PPO.

513 **Hyperparameters.** PPO clip 0.2; GAE $\lambda_{\text{GAE}} = 0.95$; discount $\gamma = 0.99$; mini-batch 64; gradient
 514 clip 0.5; value-loss coefficient 0.5; entropy coefficient 0.01 (CartPole) and 0 (others). Rollout length
 515 and PPO epochs: 512/4 for classic control (CartPole, Acrobot, Pendulum); 2,048/10 for MuJoCo.
 516 Learning rate: 3×10^{-4} for both actor and critic.

517 **Environments and horizons.** CartPole-v1 (2×10^5 steps), Acrobot-v1 (5×10^5), Pendulum-v0
 518 (3×10^5), HalfCheetah-v4, Hopper-v4, Walker2d-v4 (10^6 each). Three seeds per environment.

519 **Compute.** Classic-control PPO: ≈ 10 CPU-min per seed. MuJoCo: ≈ 4 GPU-hours per seed
 520 (single GV100).

521 A.6 Random seeds and statistical reporting

522 Seeds are integers $\{0, 1, 2, 3, 4\}$ for 5-seed runs and $\{0, 1, 2\}$ for 3-seed runs. Each seed determinis-
 523 tically initializes Python’s random, NumPy, and PyTorch RNGs as well as the Gym environment.
 524 Tables report mean $\pm$ sample standard deviation; figures shade $\pm 1\sigma$ around the mean curve. The
 525 experiments reported in this paper required no preliminary runs beyond the spectral-bound ablation
 526 in Exp E (§4.1). Total project compute: ~ 50 GPU-hours and ~ 30 CPU-hours.

527 B Structural ablations (Exp F, Exp G)

528 This appendix records the planning-task ablations referenced in §4.1. Both runs use the same 10×10
 529 stochastic-obstacle generator, training schedule, and seeds as Exp A; the only changes are to the
 530 contractive head. *Exp F* ($\lambda \equiv 0$) clamps the eigenvalue MLP’s output to zero throughout training,
 531 leaving $z^*(s) = b(s)$. *Exp G* (*dense* Λ) replaces the diagonal head with a matrix-output head, projects
 532 $\Lambda(s)$ to spectral radius $< \alpha$ by SVD clipping ($\Lambda \rightarrow U \min(\Sigma, \alpha) V^\top$), and computes $z^*(s)$ via the
 533 closed-form linear solve $(\mathbf{I} - \Lambda(s))z^* = b(s)$ at $\mathcal{O}(d^3)$ cost per state. A plain MLP baseline (no
 534 Linearizer head) is included for reference.

535 C Additional notes

536 The *Iterative MLP* baseline in §4.1 unrolls a fixed 3-layer MLP for $K = 5$ steps, with the same
 537 parameter budget as the Contractive model. The *Unconstrained* baseline is identical to Contractive
 538 except that $\Lambda(s)$ is the raw output of MLP_Λ with no $\alpha \tanh$ clamp; this is the only architectural
 539 difference, isolating the role of the spectral constraint.

Table 6: Structural ablations on the planning task. Same setup as Exp A, 3 seeds, mean $\pm$ std.

Variant	RMSE @ $K = 50$ ($\downarrow$)	Inference cost
Contractive (diagonal, ours)	<i>pending</i>	$\mathcal{O}(d)$
Dense Λ (Exp G)	<i>pending</i>	$\mathcal{O}(d^3)$
$\lambda \equiv 0$ (Exp F)	<i>pending</i>	$\mathcal{O}(d)$
Plain MLP (no Linearizer)	<i>pending</i>	$\mathcal{O}(d)$

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The four claims listed in the abstract and §1 (a contractive learned operator, $43\times$ closed-form speedup, DQN stability and benchmark gains, PPO critic-variance reduction) are each substantiated by §3 (theory) and §4 (experiments). MuJoCo PPO and Atari DQN results are still running and are explicitly marked as pending.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: §5 contains a dedicated “Limitations” paragraph covering: VIN’s grid-specific advantage, low-dimensional DQN benchmarks, CartPole’s saturation under PPO, and pending MuJoCo evaluations. Remark 1 additionally notes that the convergence guarantee is on latent z_k , not decoded RMSE.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.

- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The single theoretical result is Theorem 1 in §3.3. Its only assumption ($\alpha < 1$) is enforced by construction via the $\alpha \tanh(\cdot)$ parameterization (Eq. 7); the proof immediately follows the statement and is a one-paragraph application of the Banach fixed-point theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: App. A documents architectures (encoder/head widths, latent dim d per setting), optimizer settings, exploration schedules, replay buffer sizes, target-network update frequencies, rollout lengths, training horizons, environment versions, and per-experiment compute. Random seeds and statistical reporting are described in App. A.6.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized code archive is included with the submission and contains the full training/evaluation pipeline for all three settings (planning, DQN, PPO), pre-trained checkpoints, and a reproducible launcher script. All datasets used (Gym/Gymnasium classic-control, MuJoCo -v4, Atari) are publicly available; randomly generated grid-world maps are produced by the included generator with a fixed seed.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: §3 gives the high-level training setup, and App. A gives all per-environment hyperparameters, optimizer settings, schedules, and architectural sizes.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All result tables report mean $\pm$ sample standard deviation across 5 seeds (DQN classic control), 3 seeds (PPO, Atari), or as noted; learning-curve figures shade $\pm 1\sigma$ around the mean. Error-bar definitions and seed protocol are stated in the Reproducibility paragraph (§4.3) and App. A.6.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: App. A.1 states the hardware (single CPU for classic-control settings; one NVIDIA Quadro GV100 GPU for planning training, Atari, and MuJoCo). Approximate per-seed wall-clock times and total project compute are reported in App. A.3, A.4, A.5, and A.6.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work is foundational reinforcement-learning methodology evaluated on standard public benchmarks (Gym, MuJoCo, Atari). It involves no human subjects, no scraped data, no deployment, and no models with elevated misuse risk.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is a foundational architectural contribution to value-function approximation; it does not target any specific application and we are unaware of a direct societal impact path beyond the diffuse benefits of more stable RL training.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are RL training code and small task-specific checkpoints (CartPole/Acrobot/Pendulum/grid-world); none constitute a high-risk asset (no language model, no generative image model, no scraped data).

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All third-party benchmarks and baselines are cited at first use: Gym/OpenAI [Brockman et al., 2016], MuJoCo [Todorov et al., 2012], Atari preprocessing and DQN [Mnih et al., 2015], Double DQN [Van Hasselt et al., 2016], PPO [Schulman et al., 2017], VIN [Tamar et al., 2016]. These assets are released under standard open licenses (MIT/Apache/Nintendo-permitted research use for ALE) and are used within their license terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The accompanying anonymized code archive includes a README documenting the directory layout, dependencies, training/evaluation entry points, and the licenses of the new code (planned MIT release). Pre-trained checkpoints carry the same license.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.

- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not part of the core methodology, experiments, or analysis of this paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.